

Capstone Project >>>>

Develop a YouTube Clone Using the MERN Stack

Objective:

Create a full-stack YouTube clone where users can view and interact with videos. This project will help you understand how to build a real-world application using MongoDB, Express, React, and Node.js (MERN stack).

Start vite project:

Create new project using “**npm create vite@latest -y**”



Cd youtube_clone folder



Npm install



Npm run dev



Local server : <http://localhost:5173/>

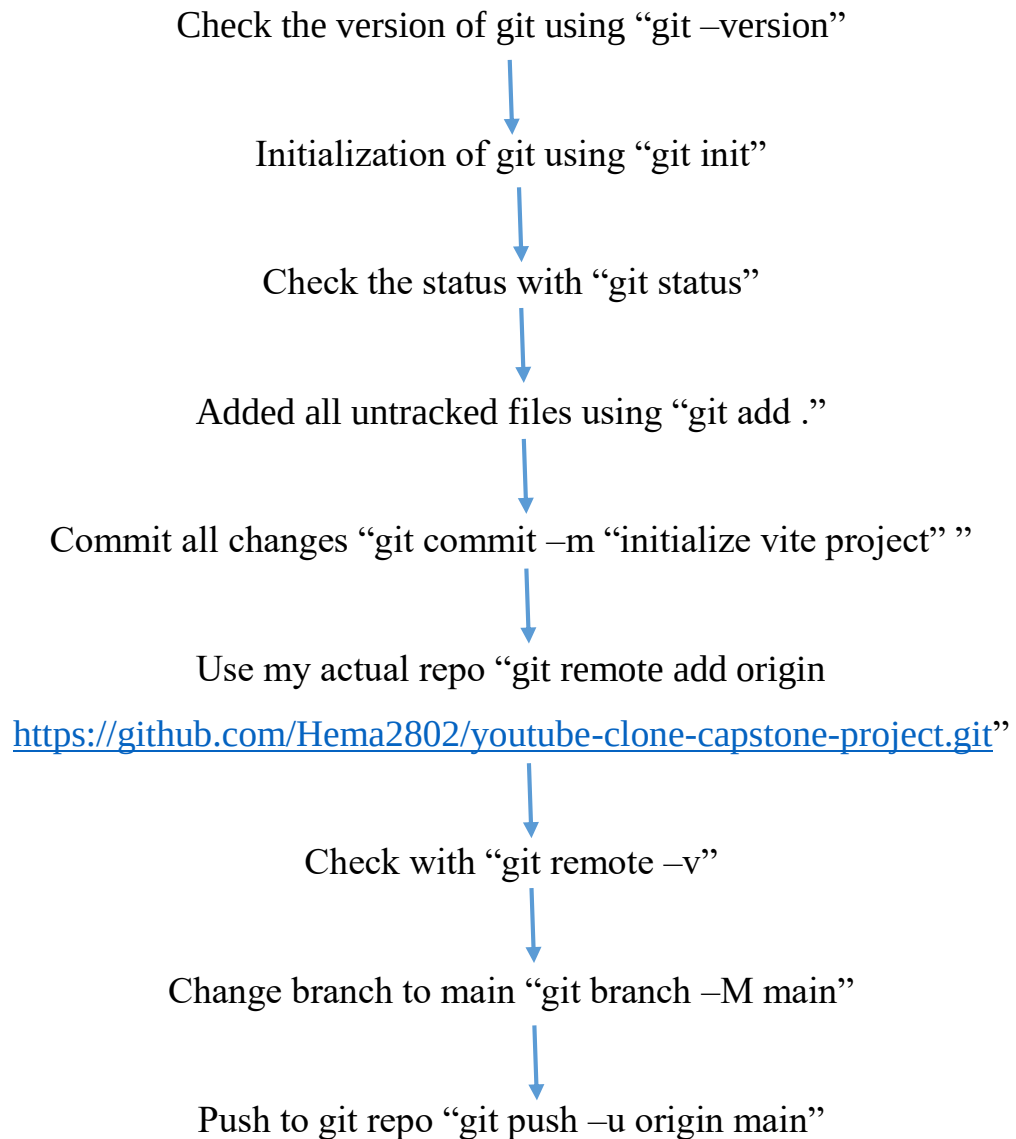


Clear default code and start your project

Installed Packages:

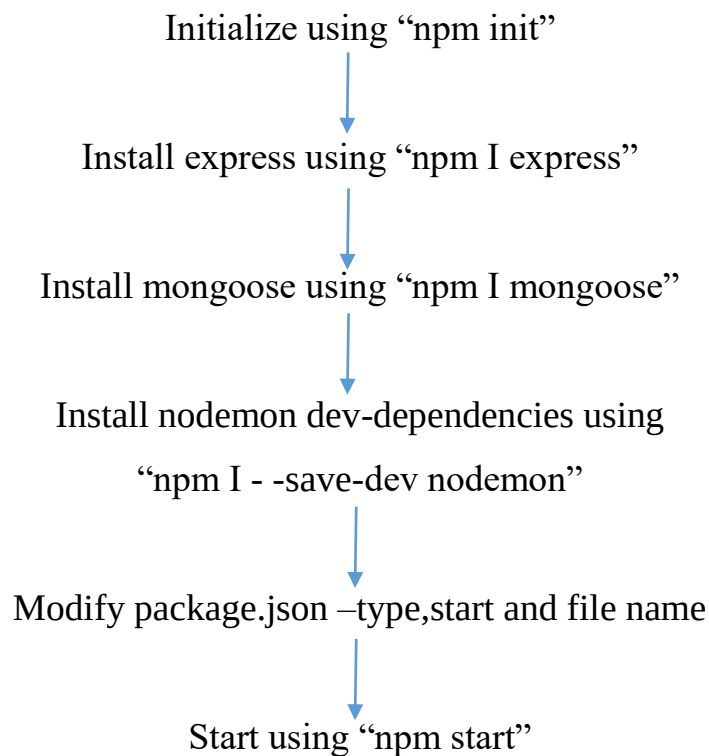
- React –router-dom using “**npm i react-router-dom**”.
- Moment package

Set Version Control System

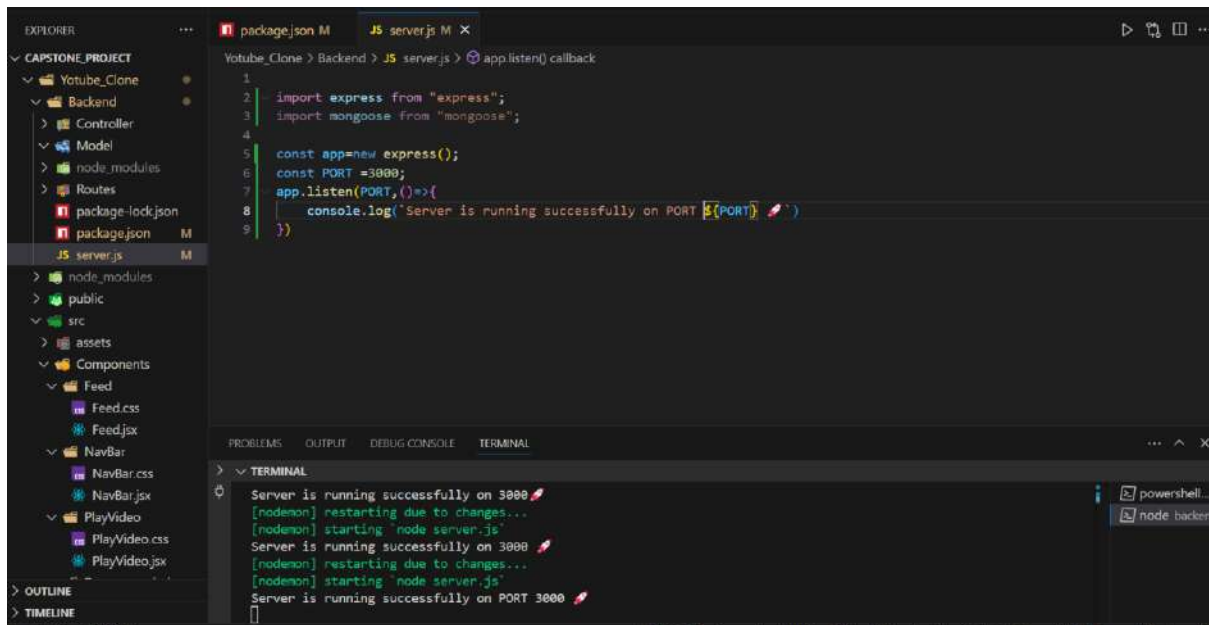


Backend for youtube clone

Set-up



Created Server –PORT 3000

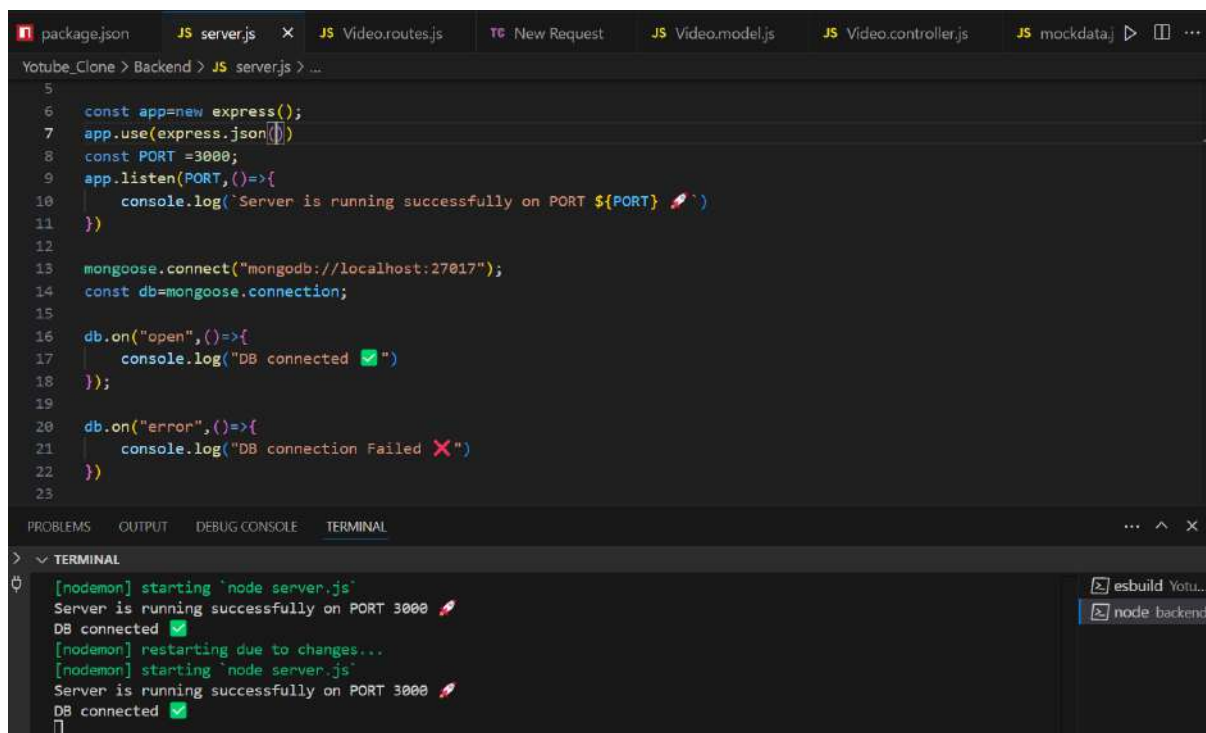


```
1
2 import express from "express";
3 import mongoose from "mongoose";
4
5 const app=new express();
6 const PORT =3000;
7 app.listen(PORT,()=>{
8   console.log('Server is running successfully on PORT ${PORT}')
9 })
```

TERMINAL

```
Server is running successfully on 3000
[nodemon] restarting due to changes...
[nodemon] starting 'node server.js'
Server is running successfully on 3000
[nodemon] restarting due to changes...
[nodemon] starting 'node server.js'
Server is running successfully on PORT 3000
```

Connected MongoDB:



The screenshot shows the VS Code editor with the `server.js` file open. The code initializes an Express application, connects to MongoDB, and logs the connection status. The terminal output shows the server starting successfully on port 3000 and connecting to the database.

```
5
6 const app=new express();
7 app.use(express.json());
8 const PORT =3000;
9 app.listen(PORT,()=>{
10   console.log(`Server is running successfully on PORT ${PORT} 🚀`)
11 })
12
13 mongoose.connect("mongodb://localhost:27017");
14 const db=mongoose.connection;
15
16 db.on("open",()=>{
17   console.log("DB connected ✅")
18 });
19
20 db.on("error",()=>{
21   console.log("DB connection Failed ❌")
22 })
23
```

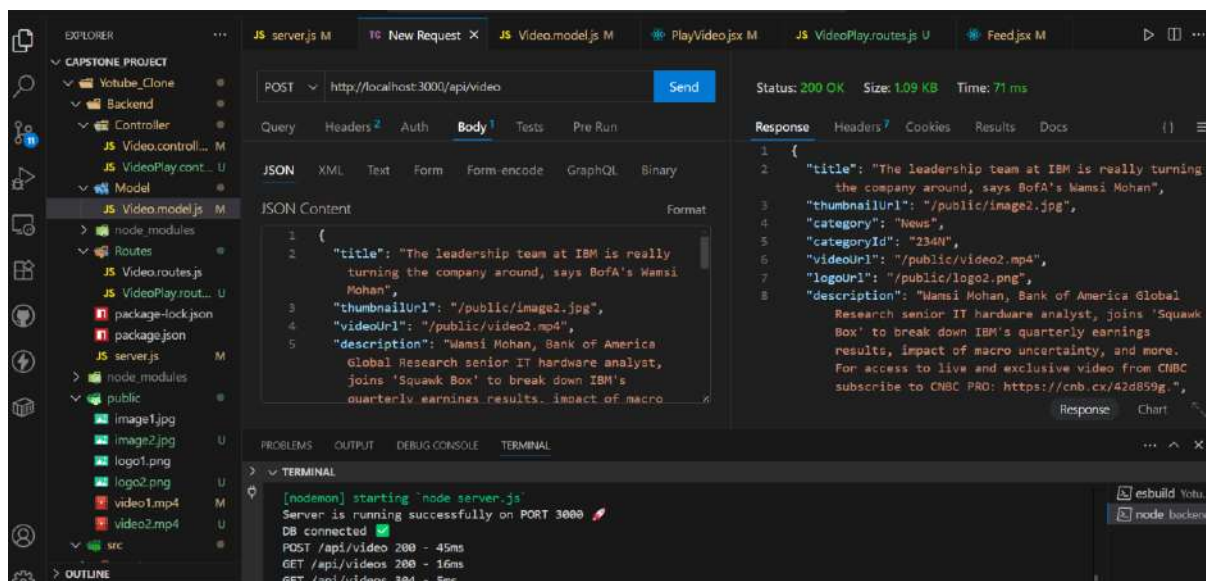
TERMINAL

```
[nodemon] starting `node server.js`
Server is running successfully on PORT 3000 🚀
DB connected ✅
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
Server is running successfully on PORT 3000 🚀
DB connected ✅
```

Created datas in mongodb:

Performed CRUD operation

Post method - Created new data



The screenshot shows the VS Code editor with the REST client open. A POST request is sent to `http://localhost:3000/api/video` with a JSON body. The response is a 200 OK status with a JSON body containing video details.

POST `http://localhost:3000/api/video` Send

Status: 200 OK Size: 1.09 KB Time: 71 ms

JSON Content

```
1 {
2   "title": "The leadership team at IBM is really turning
the company around, says BofA's Wamsi Mohan",
3   "thumbnailUrl": "/public/image2.jpg",
4   "videoUrl": "/public/video2.mp4",
5   "description": "Wamsi Mohan, Bank of America
Global Research senior IT hardware analyst, joins 'Squawk
Box' to break down IBM's quarterly earnings results. impact of macro
```

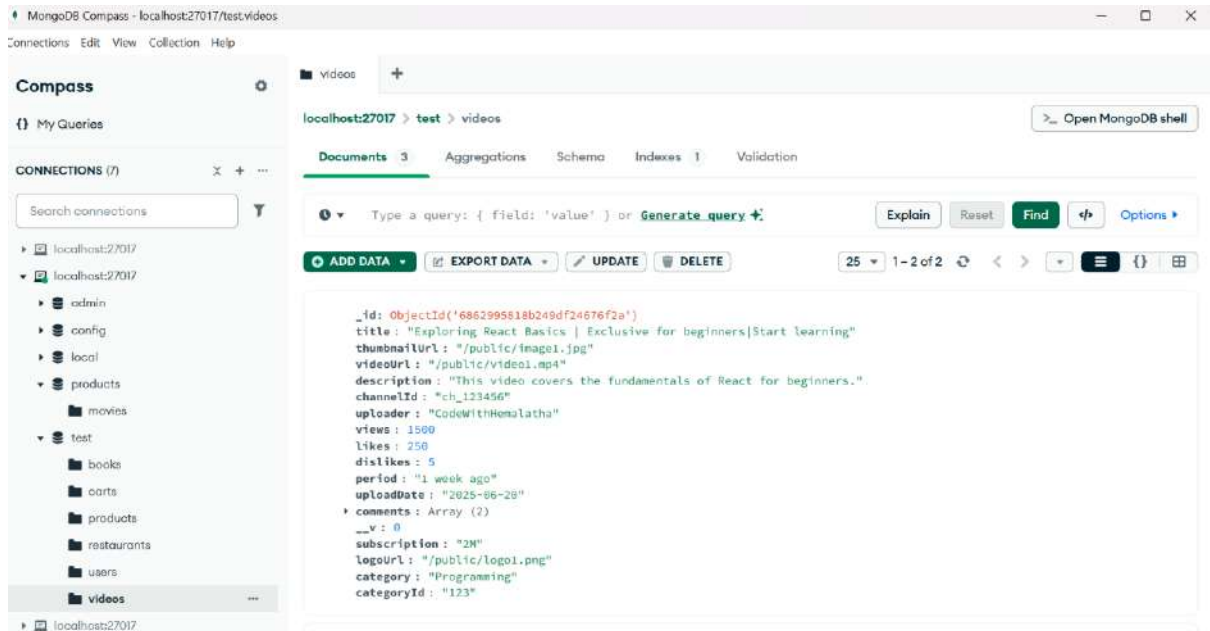
Response

```
1 {
2   "title": "The leadership team at IBM is really turning
the company around, says BofA's Wamsi Mohan",
3   "thumbnailUrl": "/public/image2.jpg",
4   "category": "News",
5   "categoryId": "234N",
6   "videoUrl": "/public/video2.mp4",
7   "logoUrl": "/public/logo2.png",
8   "description": "Wamsi Mohan, Bank of America Global
Research senior IT hardware analyst, joins 'Squawk
Box' to break down IBM's quarterly earnings results, impact of macro uncertainty, and more.
For access to live and exclusive video from CNBC
subscribe to CNBC PRO: https://cnb.cx/42d859g."}
```

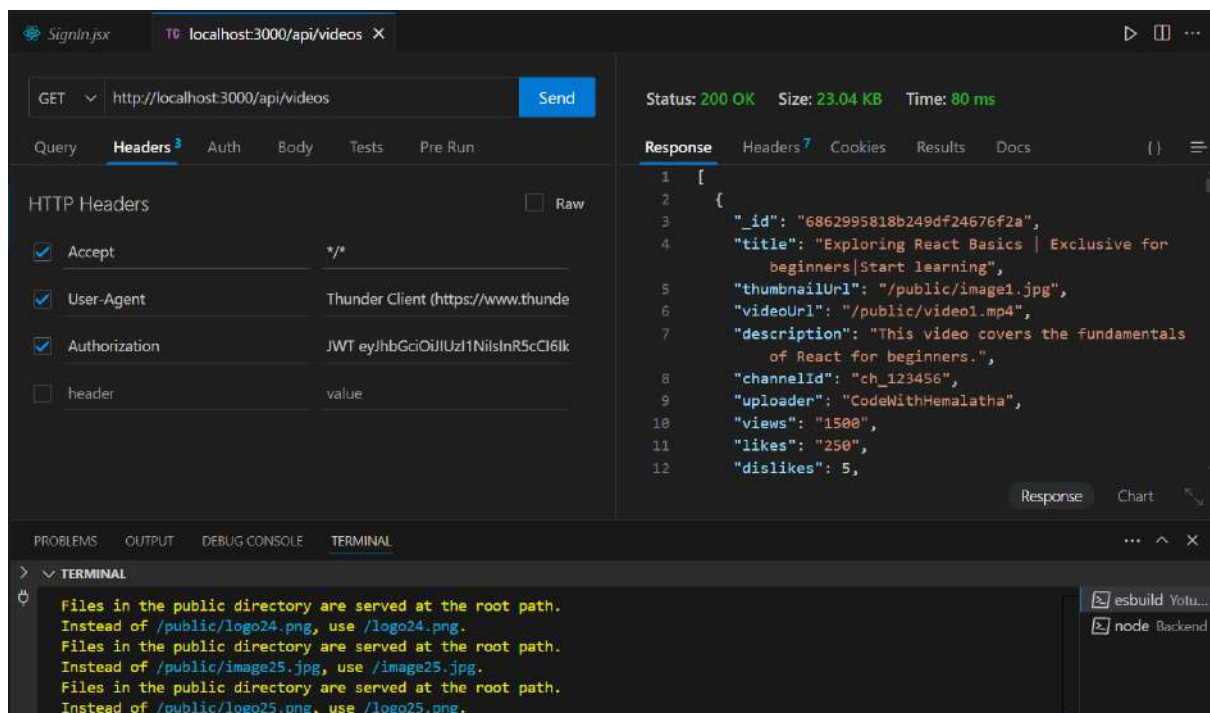
TERMINAL

```
[nodemon] starting `node server.js`
Server is running successfully on PORT 3000 🚀
DB connected ✅
POST /api/video 200 - 45ms
GET /api/videos 200 - 16ms
GET /api/videos 304 - 5ms
```

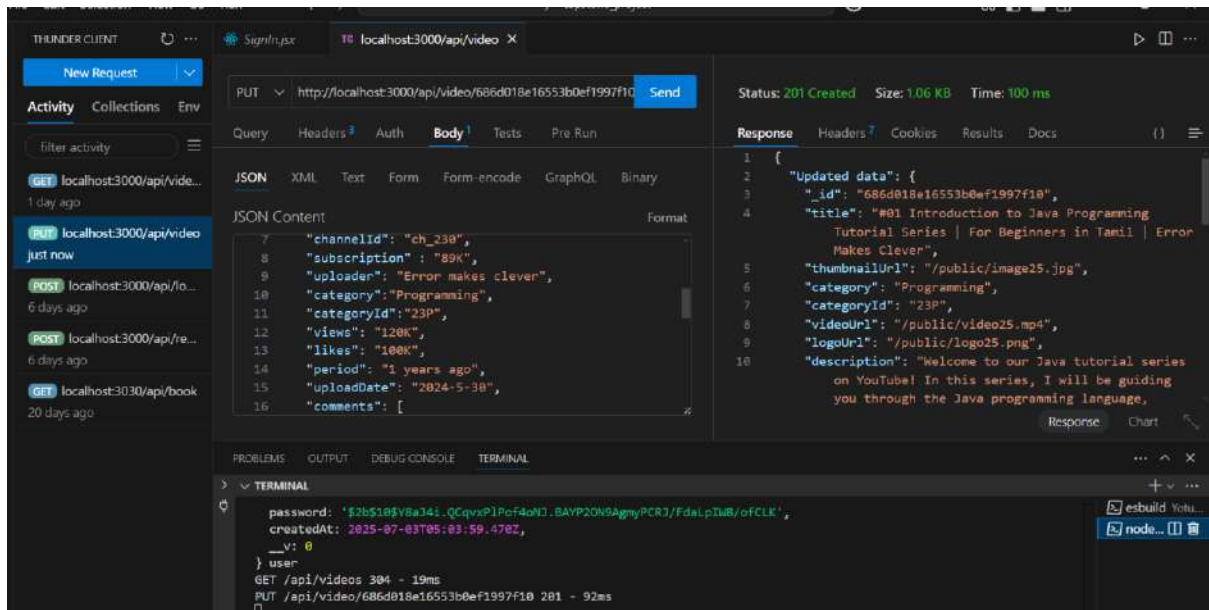
MongoDB



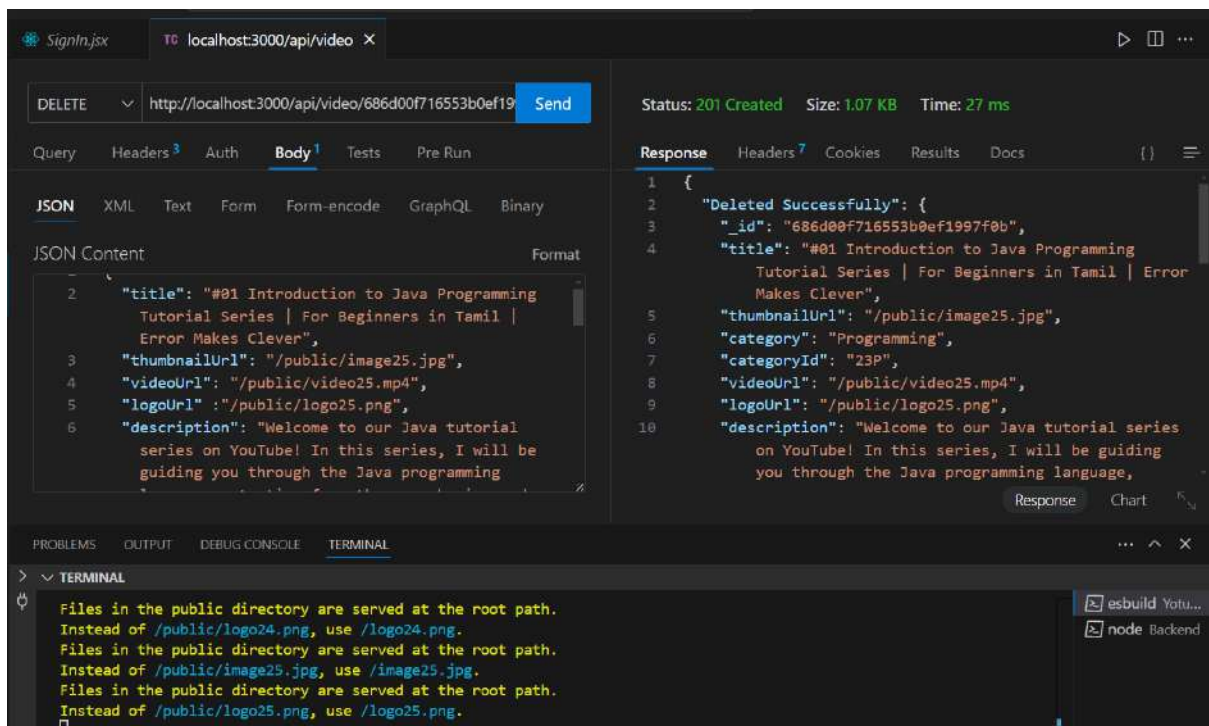
Get method – to fetch data



Put method – update existing data



Delete method – to delete unwanted data



To create API for my capstone project

- I have created Model, Controller and Routes.

CONTROLLER PART

- In this part I have created controller for user, video and playing video.

User controller:

```
TC localhost:3000/api/video X JS User.controller.js X
Youtube_Clone > Backend > Controller > JS User.controller.js > loginUser

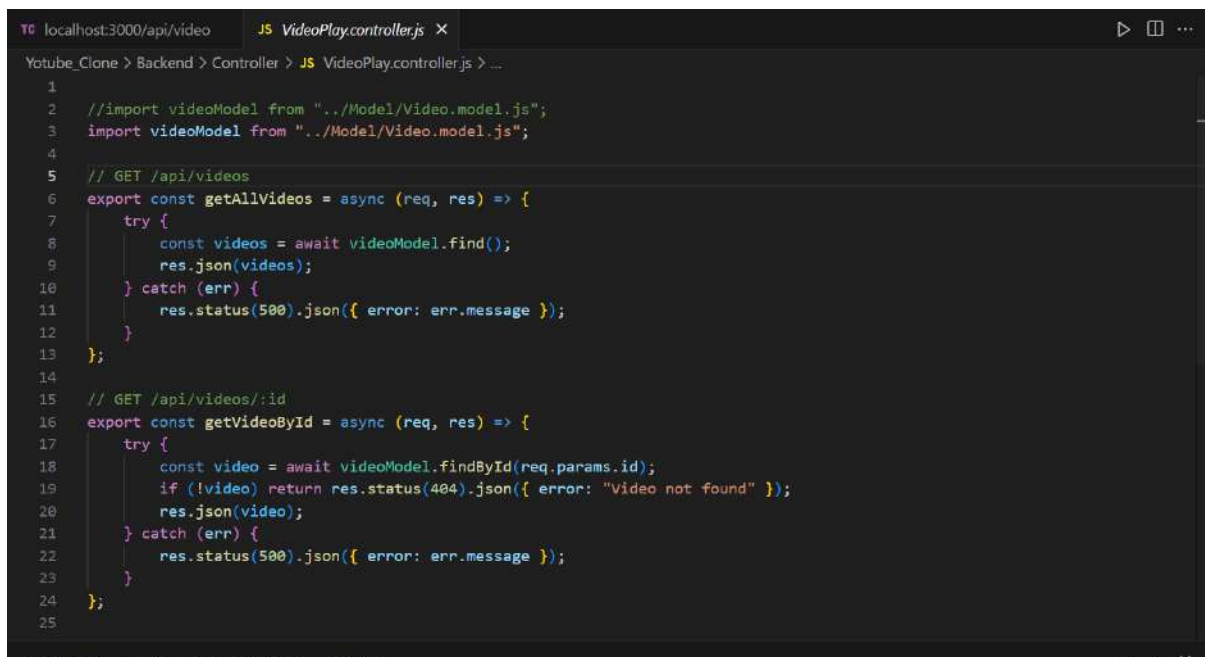
1
2 // imported part
3
4 import UserModel from "../Model/User.model.js";
5 import bcrypt from "bcrypt"
6 import jwt from "jsonwebtoken"
7
8
9 // for register the user if anyone newly visit my shoppy app
10 export function registerUser(req,res){
11     // defining req body
12     const {fullName,email,password}=req.body;
13     // find email already exist or not
14     UserModel.findOne({email})
15     .then((data)=>{
16         if(data){
17             return res.status(400).json({message:"The Email is already exist in DB."})
18         }
19
20         const newUser = new UserModel({
21             fullName,
22             email,
23             // hashing the password using bcrypt
24             password: bcrypt.hashSync(password, 10)
25         })
26     })
27 }
```

Video Controller:

```
TC localhost:3000/api/video JS Video.controller.js X
Youtube_Clone > Backend > Controller > JS Video.controller.js > fetchVideos > catch() callback

1 import videoModel from "../Model/Video.model.js";
2
3
4 export function createVideo(req,res){
5     const{title,thumbnailUrl,logoUrl,channelId,views,period,category,categoryId,description,uploader,subscription,likes,dislikes} = req.body;
6
7     const newVideo= new videoModel({
8         title : title,
9         channelId:channelId,
10        views:views,
11        period:period,
12        subscription:subscription,
13        category:category,
14        categoryId:categoryId,
15        thumbnailUrl:thumbnailUrl,
16        logoUrl:logoUrl,
17        description:description,
18        uploader:uploader,
19        likes:likes,
20        dislikes:dislikes,
21        uploadDate:uploadDate,
22        comments:comments,
23        videoUrl:videoUrl
24    })
25 }
```

Playing video controller:



```
1
2 //import videoModel from "../Model/Video.model.js";
3 import videoModel from "../Model/Video.model.js";
4
5 // GET /api/videos
6 export const getAllVideos = async (req, res) => {
7   try {
8     const videos = await videoModel.find();
9     res.json(videos);
10  } catch (err) {
11    res.status(500).json({ error: err.message });
12  }
13 };
14
15 // GET /api/videos/:id
16 export const getVideoById = async (req, res) => {
17   try {
18     const video = await videoModel.findById(req.params.id);
19     if (!video) return res.status(404).json({ error: "Video not found" });
20     res.json(video);
21   } catch (err) {
22     res.status(500).json({ error: err.message });
23   }
24 };
25
```

MODEL PART

- In this part ,I have created model for user and videos
- I have created schema and proper validation .

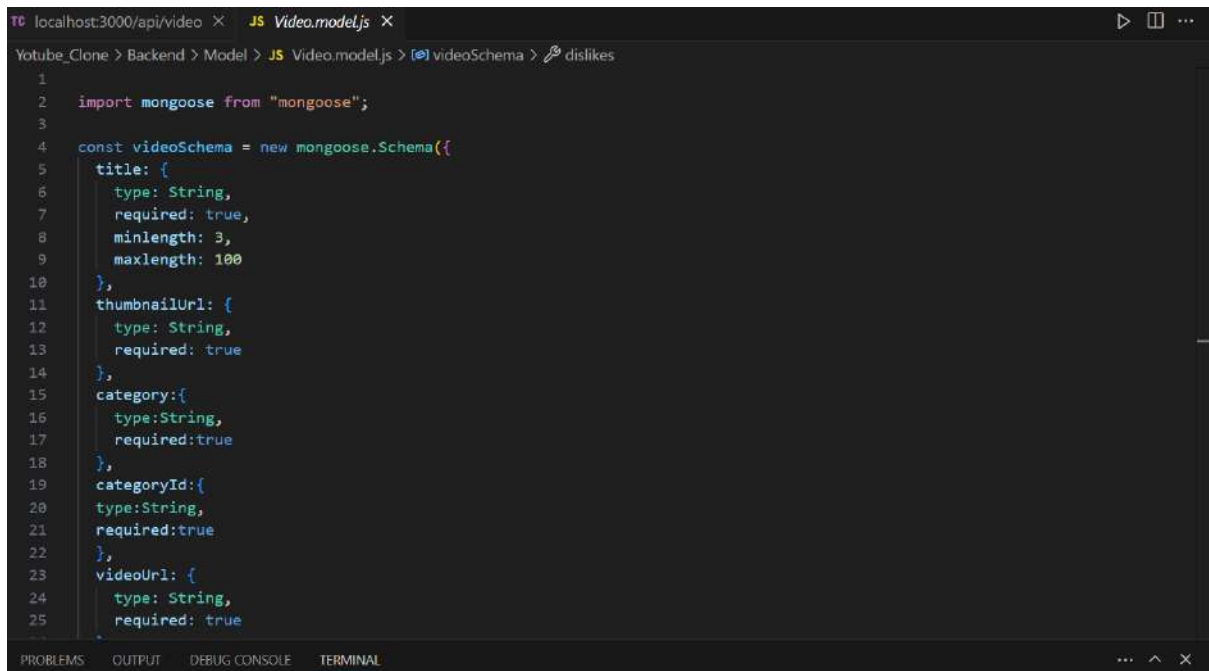
User Model:

```
import mongoose from 'mongoose';

// created schema for users

const userSchema = new mongoose.Schema({
  fullName: {
    type: String,
    required: [true, "User name is required*"],
    minlength: 3,
  },
  email: {
    type: String,
    required: [true, "Email is required*"],
    unique: true,
    match: [/^\S+@\S+\.\S+$/, "Invalid email format"],
  },
  password: {
    type: String,
    required: [true, "Password is required*"],
    minlength: 3,
  },
  createdAt: {
```

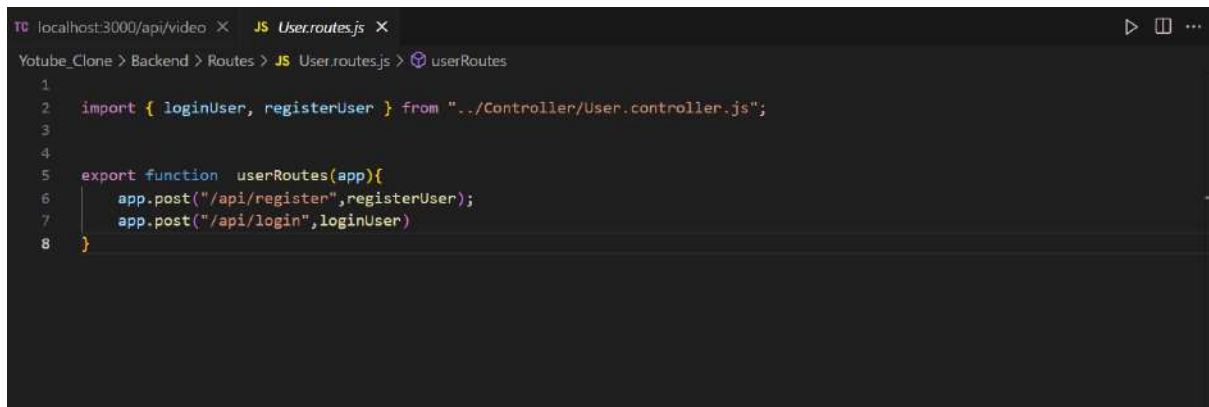

Videos Model:



```
1
2 import mongoose from "mongoose";
3
4 const videoSchema = new mongoose.Schema({
5   title: {
6     type: String,
7     required: true,
8     minlength: 3,
9     maxlength: 100
10  },
11  thumbnailUrl: {
12    type: String,
13    required: true
14  },
15  category: {
16    type: String,
17    required: true
18  },
19  categoryId: {
20    type: String,
21    required: true
22  },
23  videoUrl: {
24    type: String,
25    required: true
26  }
27 });
```

ROUTES PART

- In this part, I have created routes for videos
- Created routes for authentication



```
1
2 import { loginUser, registerUser } from "../Controller/User.controller.js";
3
4
5 export function userRoutes(app){
6   app.post("/api/register", registerUser);
7   app.post("/api/login", loginUser);
8 }
```

For videos

```
Yotube_Clone > Backend > Routes > JS Video.routes.js > ...
1
2 import { createVideo, deleteVideo, fetchVideos, searchVideo, updateVideoData } from "../Controller/Video.controller.js";
3 import { VerifyToken } from "../Middleware/VerifyToken.js";
4
5 export function routes(app) {
6     app.post("/api/video", VerifyToken, createVideo);
7     app.get("/api/videos", VerifyToken, fetchVideos);
8     app.put("/api/video/:id", updateVideoData);
9     app.delete("/api/video/:id", deleteVideo);
10    app.get("/search", searchVideo);
11 }
```

For video-play

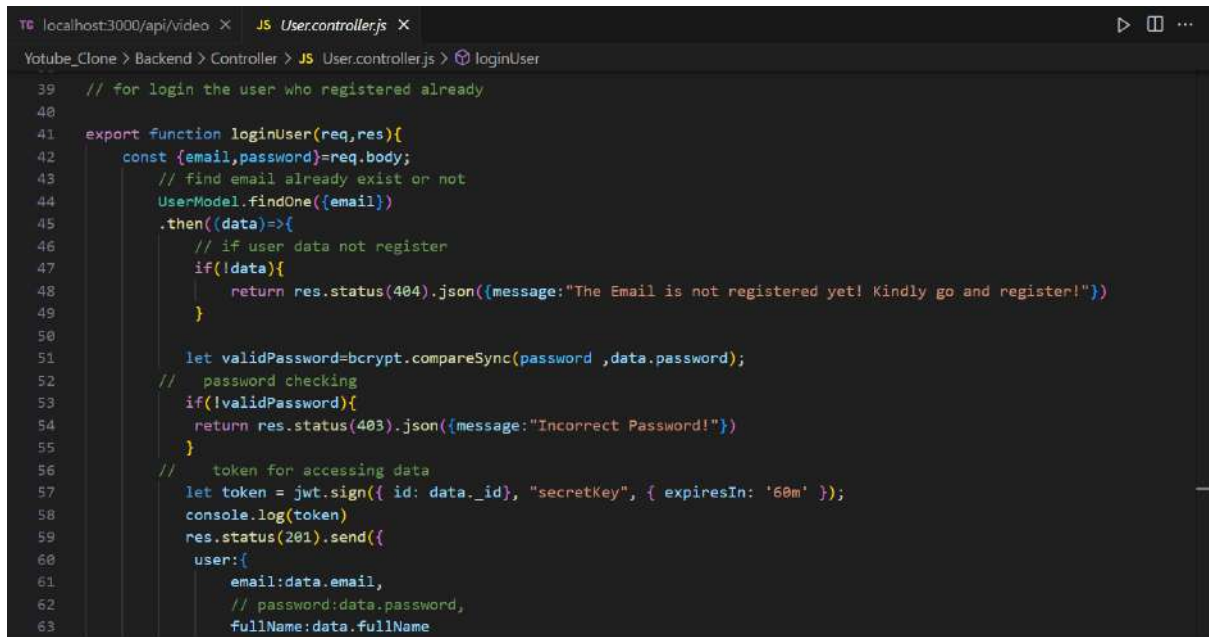
```
Yotube_Clone > Backend > Routes > JS VideoPlay.routes.js > ...
1 import { getAllVideos, getVideoById } from "../Controller/VideoPlay.controller.js";
2 import { VerifyToken } from "../Middleware/VerifyToken.js";
3
4
5
6
7 export function videoRoutes(app){
8     // GET all videos
9     app.get("/api/videos", VerifyToken, getAllVideos);
10
11     // GET video by ID
12     app.get("/api/videos/:id", VerifyToken, getVideoById);
13 }
14
15
16
17
```

Authentication

Login by users

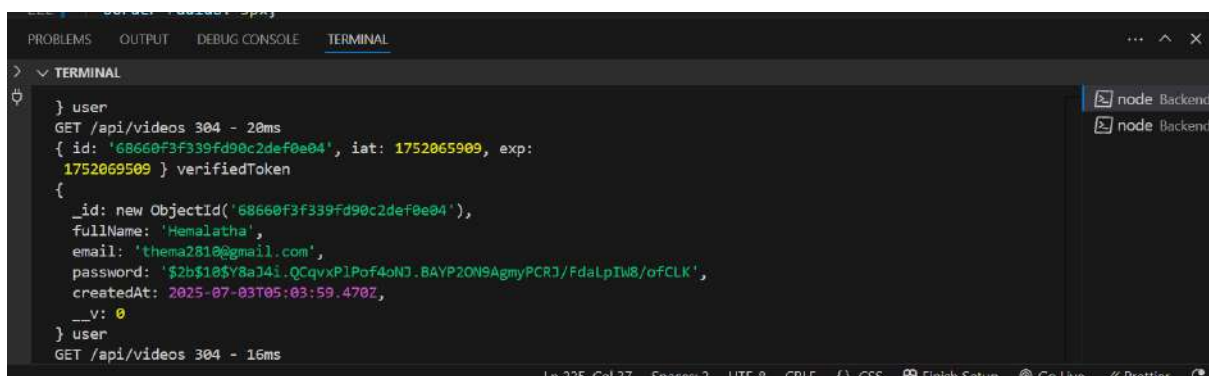
- For login the user, I have created the route in model part – “/api/login”, loginUser.
- I have written the logic to login the page in controller part.
- Validate the email and password with proper error handling.
- Validate the password using bcrypt.

- Check with thunder-client.



The screenshot shows a VS Code editor window with the file path 'Youtube_Clone > Backend > Controller > JS User.controller.js' and the function 'loginUser'. The code is as follows:

```
39 // for login the user who registered already
40
41 export function loginUser(req,res){
42   const {email,password}=req.body;
43   // find email already exist or not
44   UserModel.findOne({email})
45   .then((data)=>{
46     // if user data not register
47     if(!data){
48       return res.status(404).json({message:"The Email is not registered yet! Kindly go and register!"})
49     }
50
51     let validPassword=bcrypt.compareSync(password ,data.password);
52     // password checking
53     if(!validPassword){
54       return res.status(403).json({message:"Incorrect Password!"})
55     }
56     // token for accessing data
57     let token = jwt.sign({ id: data._id}, "secretKey", { expiresIn: '60m' });
58     console.log(token)
59     res.status(201).send({
60       user:{
61         email:data.email,
62         // password:data.password,
63         fullName:data.fullName
```



The screenshot shows the terminal output of a GET request to /api/videos. The output is as follows:

```
> } user
GET /api/videos 304 - 20ms
{ id: '68660f3f339fd90c2def0e04', iat: 1752065909, exp:
1752069509 } verifiedToken
{
  _id: new ObjectId('68660f3f339fd90c2def0e04'),
  fullName: 'Hemalatha',
  email: 'thema2810@gmail.com',
  password: '$2b$10$Y8aJ4i.QCqvXp1Pof4oNJ.BAYP2ON9AgmyPCRJ/FdaLpIW8/ofCLK',
  createdAt: 2025-07-03T05:03:59.470Z,
  __v: 0
} user
GET /api/videos 304 - 16ms
```

Thunder-client Response

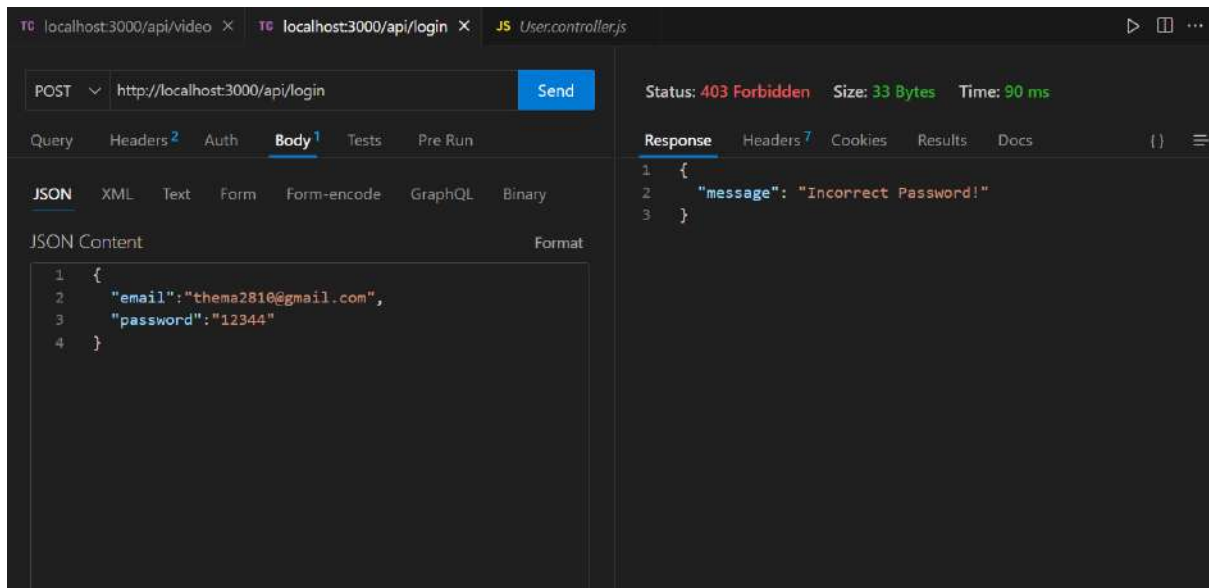
Thunder-client interface showing a successful POST request to `http://localhost:3000/api/login`. The status is **201 Created**, size is **251 Bytes**, and time is **79 ms**. The response is a JSON object:

```
{
  "user": {
    "email": "thema281@gmail.com",
    "fullName": "Hemalatha"
  },
  "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6IjY4NjYwZjNmMzMSZmQ5MGMyZGVmMGUwNCIsIm1hdCI6MTc1NTk3MzUzNCwiZXhwIjoxNzUxOTc3MTM0fQ.tjp949T03f8pqnFmbZKixiV0r53FEHxQ18G4MzSZag"
```

Error handling

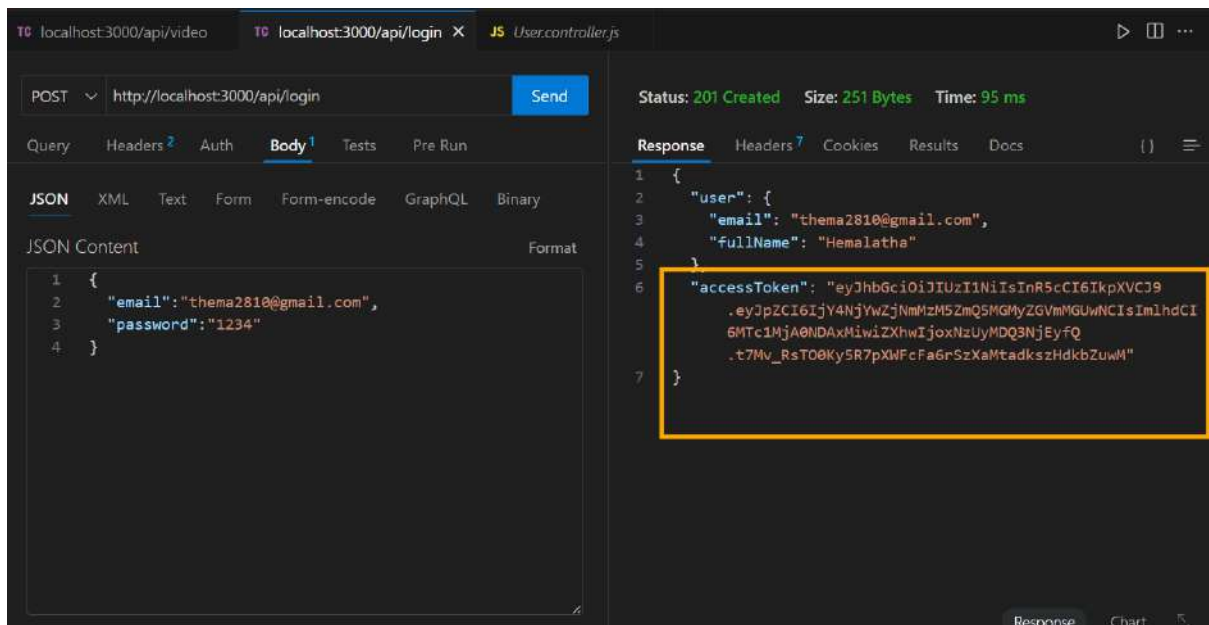
Thunder-client interface showing a failed POST request to `http://localhost:3000/api/login`. The status is **404 Not Found**, size is **70 Bytes**, and time is **26 ms**. The response is a JSON object:

```
{
  "message": "The Email is not registered yet! Kindly go and register!"
}
```



Authorization

- JSON web token is installed using “npm i jsonwebtoken”
- I have created a access_token.



Verify Token

```
import jwt from "jsonwebtoken"
import UserModel from "../Model/User.model.js";

// middleware
export function VerifyToken(req,res,next){
  if(req.headers && req.headers.authorization && req.headers.authorization.split(" ")[0]=="JWT")
  {
    // jwt method to verify the token
    jwt.verify(req.headers.authorization.split(" ")[1], 'secretKey',

    function(err, verifiedToken) {
      // if token is incorrect - error message
      if(err){
        return res.status(404).json({message:"Invalid JWT token.Kindly recheck your token!"})
      }
      // console the verified token
      console.log(verifiedToken,'verifiedToken')

      UserModel.findById(verifiedToken.id)
      .then((user)=>{

        console.log(user,"user");
        req.user=user;
      })
    })
  }
}
```


UI Components to show case

Assets section:

- In assets folders ,I have added needed images and logos.

Public folder:

- I have added needed images ,videos and logos to make video card for youtube clone which are inserted in API

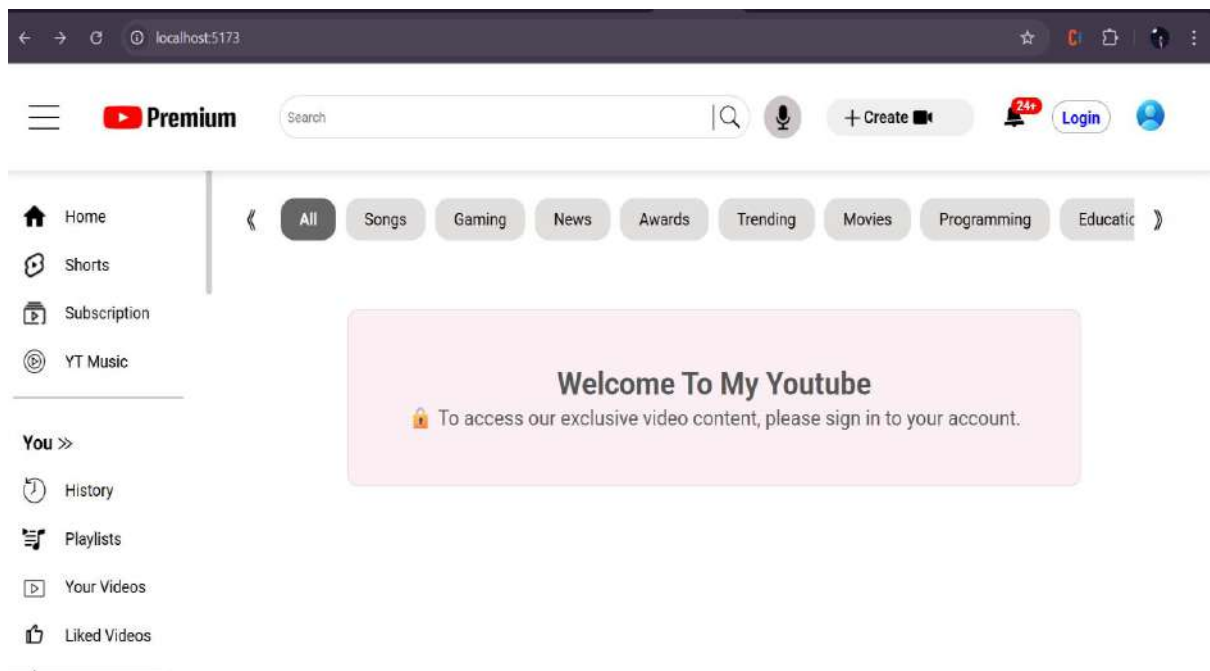
Components folder Description

S.No	Components	Description
1.	Home	Controls the overall front page layout. Combines multiple components (like NavBar, SideBar, Feed, etc.) and includes its own JSX and CSS files.
2.	NavBar	Contains the top navigation bar elements including the menu toggle, search bar, microphone icon, create/upload button, and login/logout functionality.
3.	SideBar	Provides vertical navigation links such as Home, Explore, Subscriptions, and Settings. Helps users quickly access different sections of the app.
4.	SubTitleBar	Displays the video category filters (e.g., All, Sports, Music, News) allowing users to filter the video feed based on selected categories.

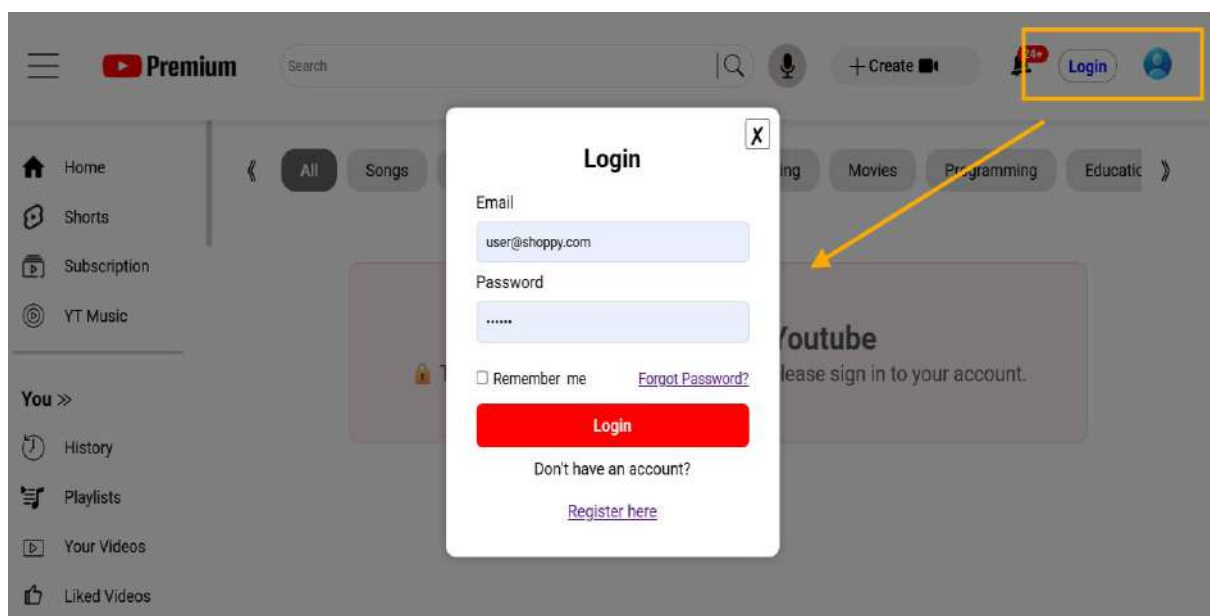
5.	SignIn	Manages user login functionality with exclusive styling. Handles authentication logic and UI for signing in.
6.	CreateAccount	Allows new users to register an account. After successful account creation, users are able to log in and access the app's content.
7.	Feed	Fetches video data from the backend/API and displays it in a grid layout as video cards for users to browse.
8.	PlayVideo	Displays a detailed view of a selected video, including the video player, description, likes/comments, and other metadata. Also includes the Recommended component.
9.	Recommended	Shows a list of recommended videos on the right side of the PlayVideo page, based on the currently playing video.
10.	CreateChannel	Provides a form for users to create their own channel. Accessed by clicking the "Create" button in the NavBar.
11.	ChannelPage	Displays the user's created channel page including tabs like Home, Videos, Playlists, and more. Shows uploaded videos and channel details.

Screen Shot of Project

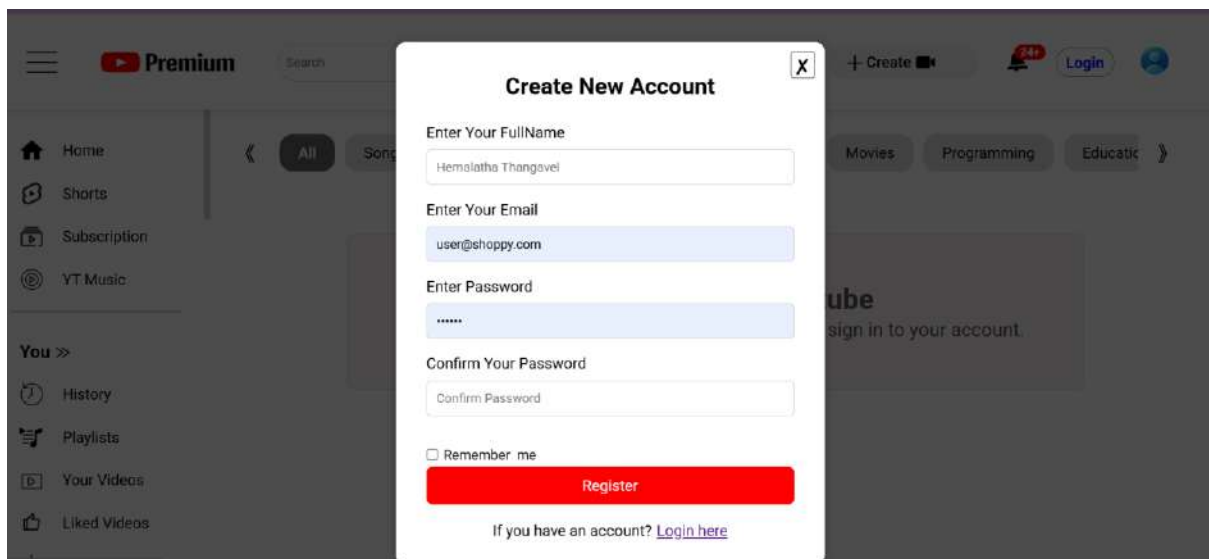
Front View



Login page



Create Account for new Users

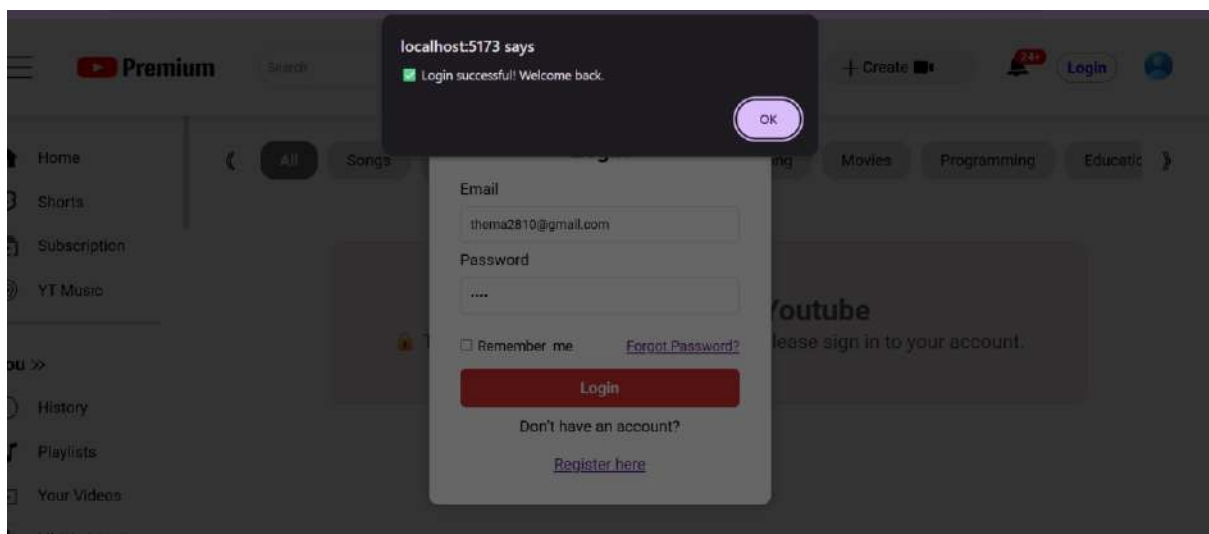


The screenshot shows a 'Create New Account' modal window overlaid on a YouTube Premium interface. The modal contains the following fields and elements:

- Create New Account** (Title)
- Enter Your FullName**: Input field with the text 'Hemalatha Thangavel'.
- Enter Your Email**: Input field with the text 'user@shoppy.com'.
- Enter Password**: Input field with masked characters '.....'.
- Confirm Your Password**: Input field with the placeholder text 'Confirm Password'.
- ☐ Remember me
- Register** (Red button)
- [Login here](#) (Link)

The background interface includes a 'Premium' logo, a search bar, and navigation links for Home, Shorts, Subscription, and YT Music.

Alert after login



The screenshot shows a login process on a YouTube Premium interface. A dark alert box is displayed at the top, indicating a successful login. Below it, a login modal is visible.

Alert:

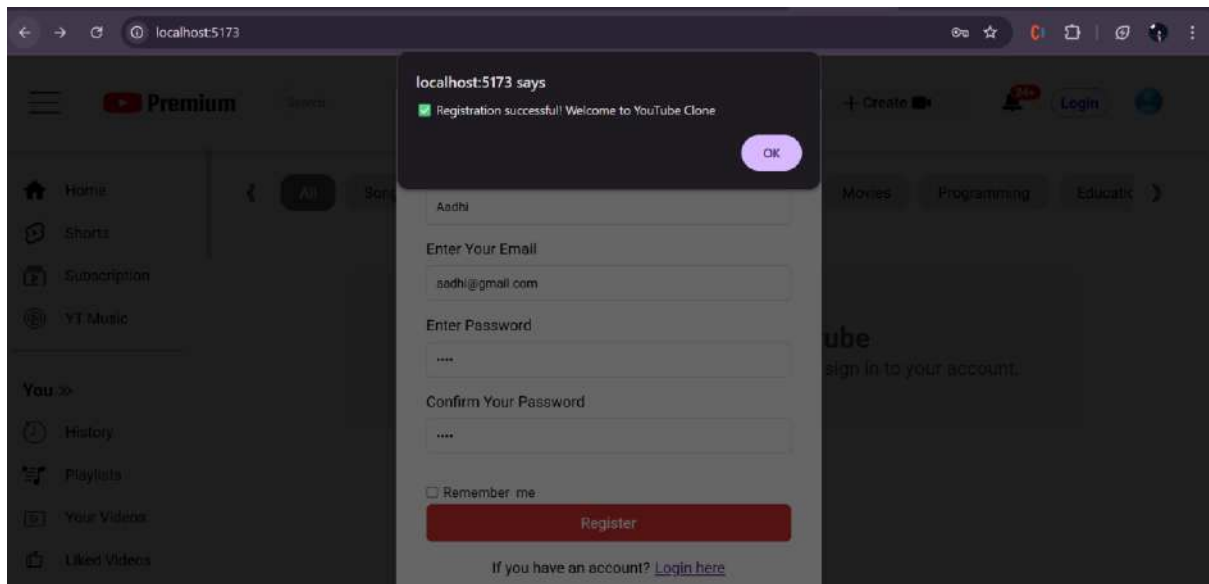
- localhost:5173 says
- Login successful! Welcome back.
- OK button

Login Modal:

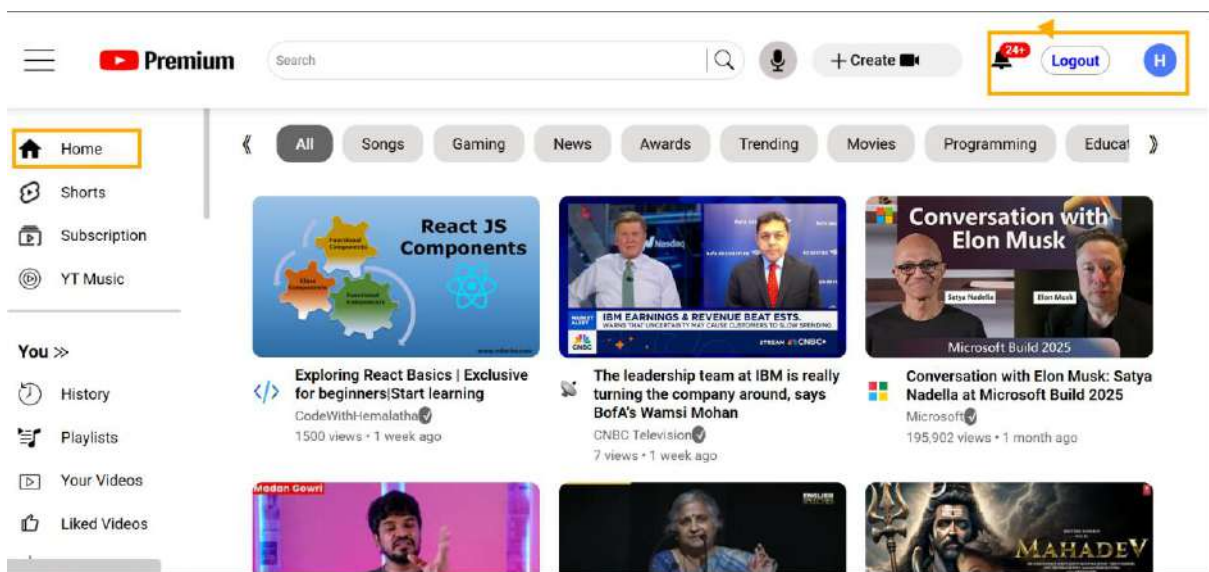
- Email**: Input field with the text 'thema2810@gmail.com'.
- Password**: Input field with masked characters '....'.
- ☐ Remember me
- [Forgot Password?](#) (Link)
- Login** (Red button)
- Don't have an account? [Register here](#) (Link)

The background interface includes a 'Premium' logo, a search bar, and navigation links for Home, Shorts, Subscription, and YT Music.

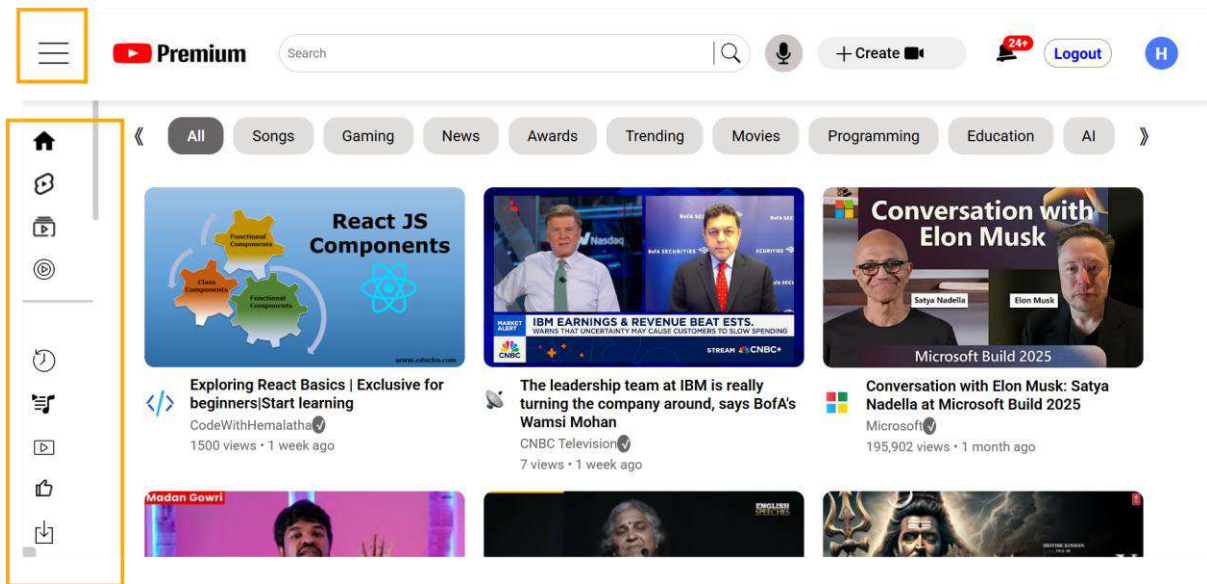
After register



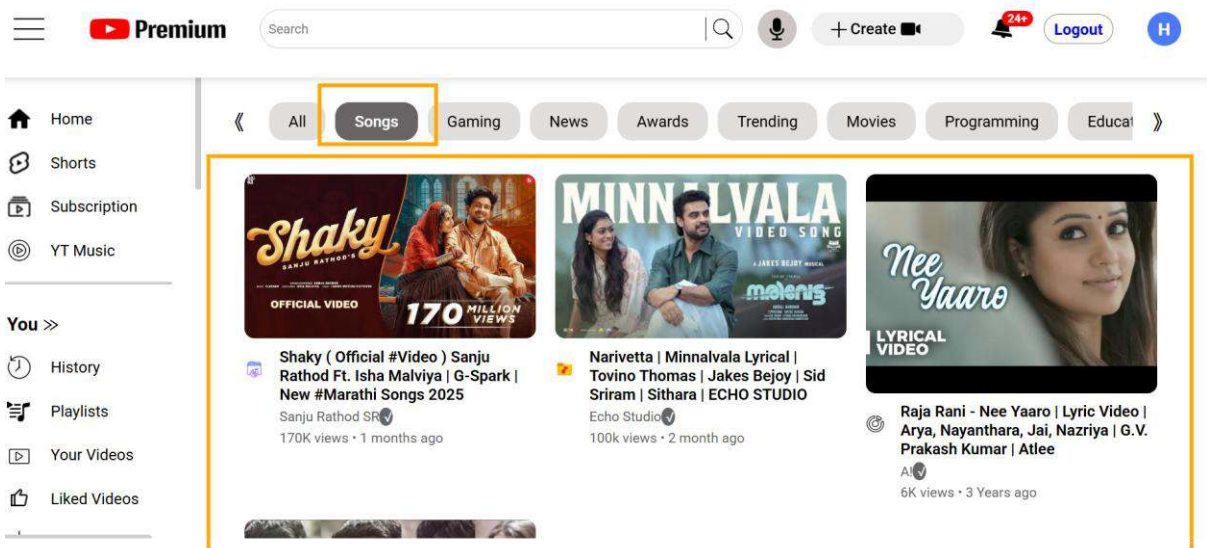
Videos Visibility



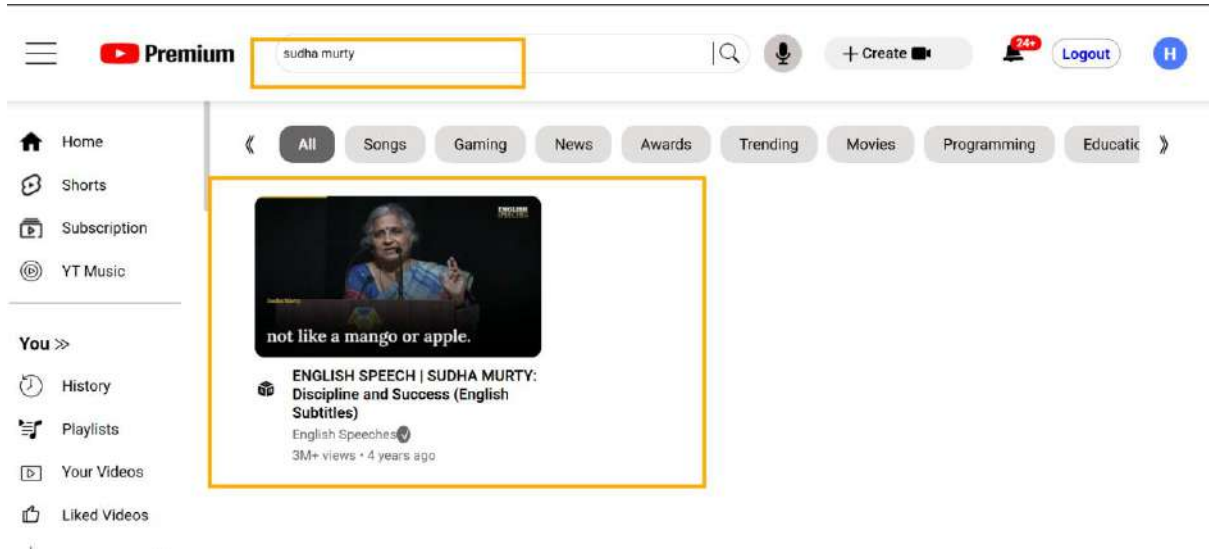
When you click menu-bar functionality



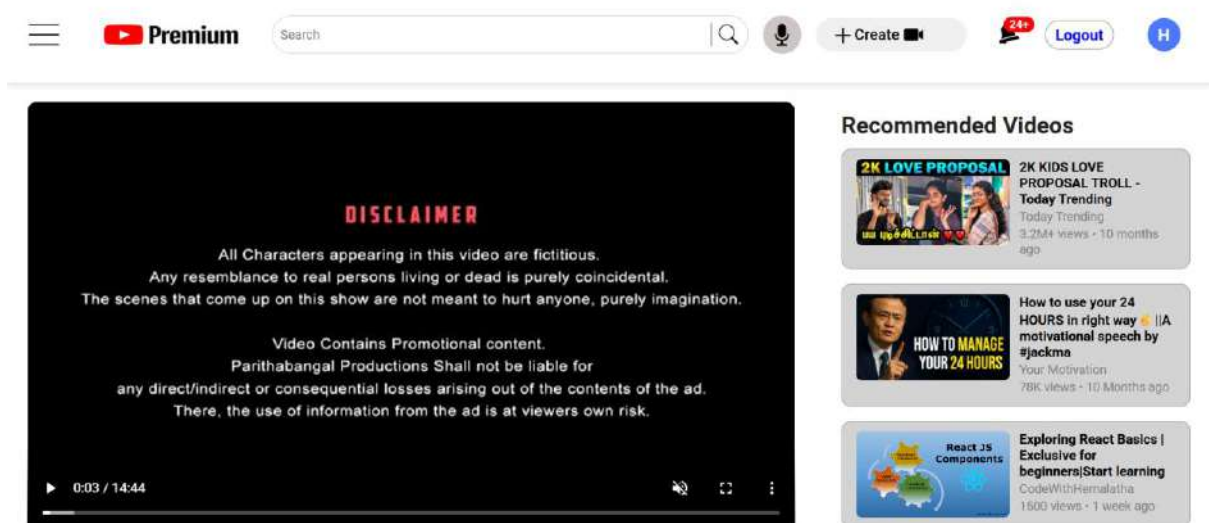
Category-wise filtration



Search Functionality



Play Video



Add comments

Premium

Search

+ Create

24

Logout

H

4 comments

Your Name

Add a comment...

Post

hema

Just now

good joke

0

TamilLaughs

10 days ago

Unmaiya supermarket-la naanum ipdi dhaan paavangal! 🍌

250

We Are Not SAFE?! 🇮🇳 | Madan Gowri | MG Squad

Madhan Gowri

277K views • 10 days ago

ENGLISH SPEECH | SUDHA MURTY: Discipline and Success (English Subtitles)

English Speeches

3M+ views • 4 years ago

Mahadev - Trailer | Hrithik Roshan | Aishwarya Rai | Amitabh Bachchan | New Movie Trailer 2025

Unreel Films

3M+ views • 4 months ago

Shaky (Official #Video)

Sani Rathod Ft. Isha

Users can edit their comments

hema

Just now

good joke

Save ✓

0

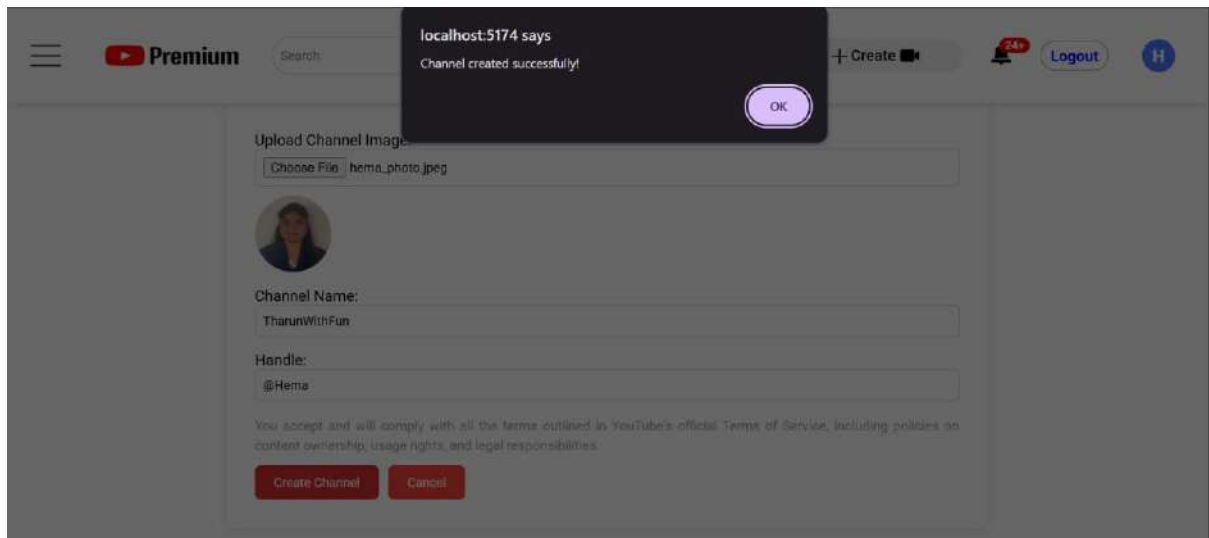
Recommended Videos

The image shows a YouTube interface. At the top, there is a navigation bar with a menu icon, a 'Premium' badge, a search bar, a microphone icon, a '+ Create' button, a notification bell with '24+', a 'Logout' button, and a user profile icon 'H'. Below the navigation bar, the main content area is divided into two sections. On the left is a video player showing a video with the text 'WELCOME @YOURMOTIVATION And Today I Want To Talk To You About'. The video has a progress bar at 0:02 / 8:33. On the right is a 'Recommended Videos' sidebar. It contains three video thumbnails with their titles and view counts:

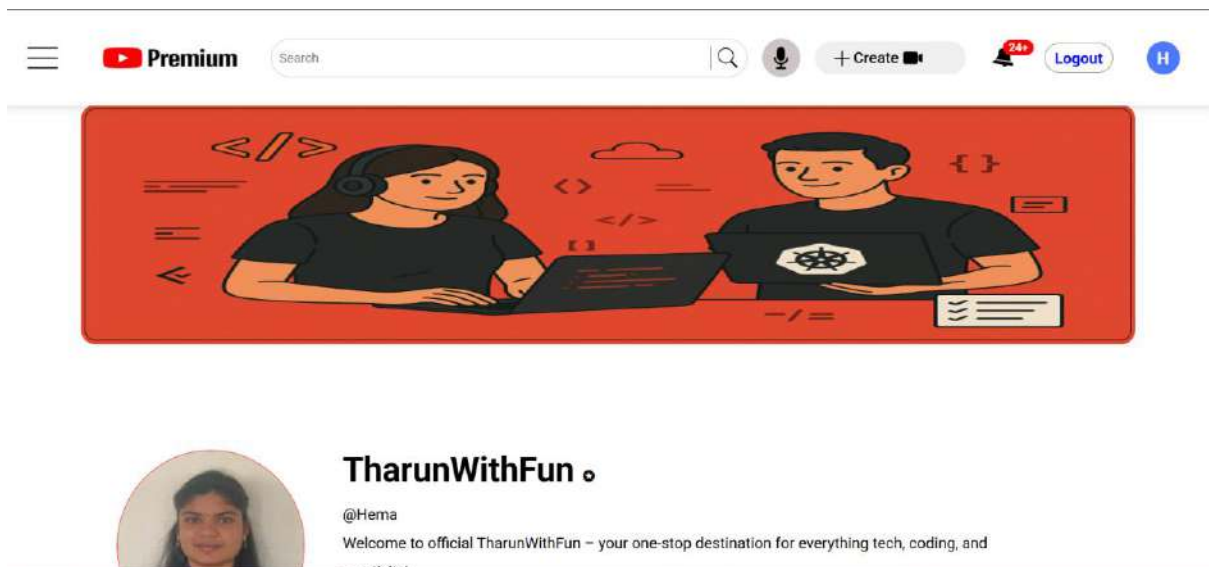
- 2K LOVE PROPOSAL**: 2K KIDS LOVE PROPOSAL TROLL - Today Trending, Today Trending, 3.2M+ views • 10 months ago
- C Mart Paavangal | Parithabangal**: Parithabangal, 215000 views • 4 weeks ago
- React JS Components**: Exploring React Basics | Exclusive for beginners! Start learning, CodeWithHemalatha, 1500 views • 1 week ago

Users can create a Channel

The image shows the 'Create Your Channel' form on YouTube. At the top, there is a navigation bar with a menu icon, a 'Premium' badge, a search bar, a microphone icon, a '+ Create' button (highlighted with a yellow box and an arrow), a notification bell with '24+', a 'Logout' button, and a user profile icon 'H'. Below the navigation bar, the main content area is titled 'Create Your Channel'. It features a blue circular profile picture placeholder. Below the placeholder, there are three input fields: 'Upload Channel Image:' with a 'Choose File' button and 'No file chosen' text, 'Channel Name:', and 'Handle:' with the text '@yourhandle'. At the bottom, there is a small disclaimer: 'You accept and will comply with all the terms outlined in YouTube's official Terms of Service, including policies on content ownership, usage rights, and legal responsibilities.'



Channel page





TharunWithFun

@Hema

Welcome to official TharunWithFun – your one-stop destination for everything tech, coding, and creativity!

New videos every Tuesday and Friday

6.00pm - 7.00pm

Subscribe

Upload Content

Home Videos Sports Live PlayLists Community

Upload videos

Premium

Search

+ Create

24+

Logout

H

TharunWithFun

@Hema

Welcome to official TharunWithFun – your one-stop destination for everything tech, coding, and creativity!

New videos every Tuesday and Friday

6.00pm - 7.00pm

Subscribe Upload Content

Upload Your Video Content

Video Title

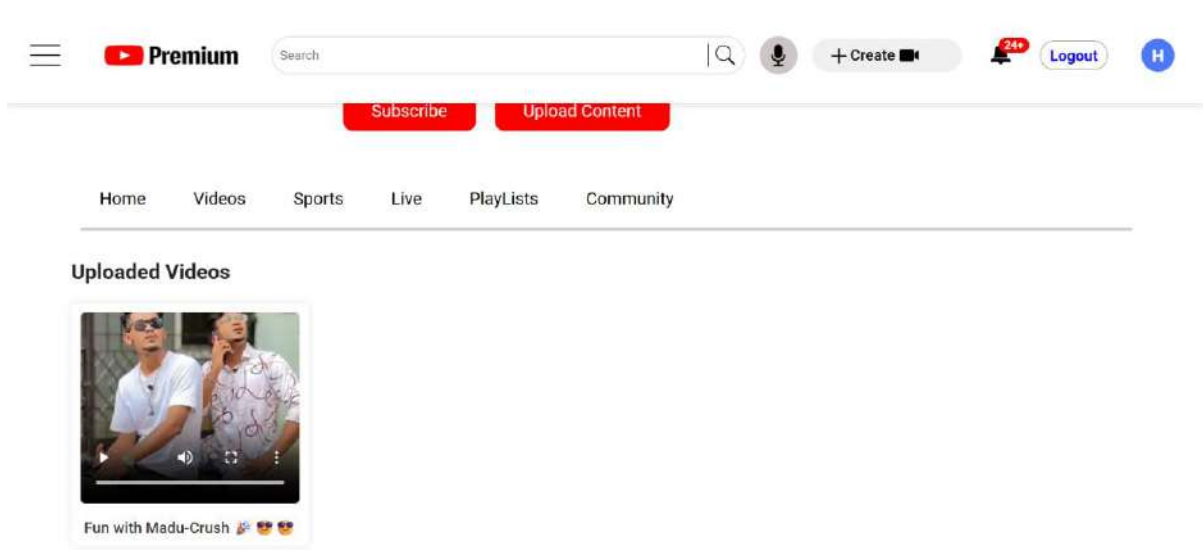
Choose File No file chosen

Upload

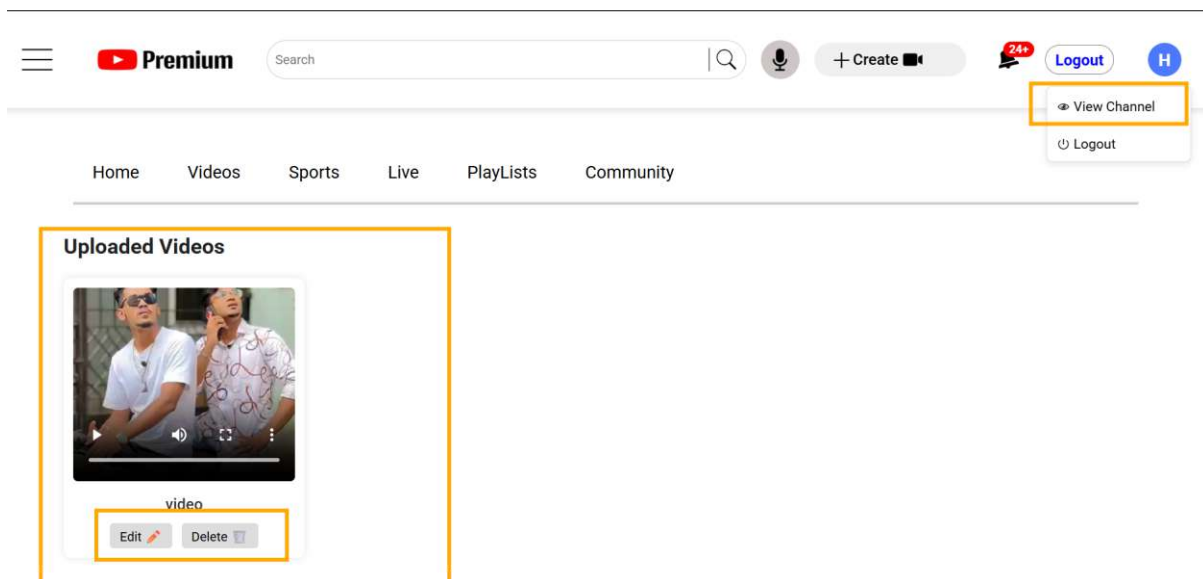
Cancel

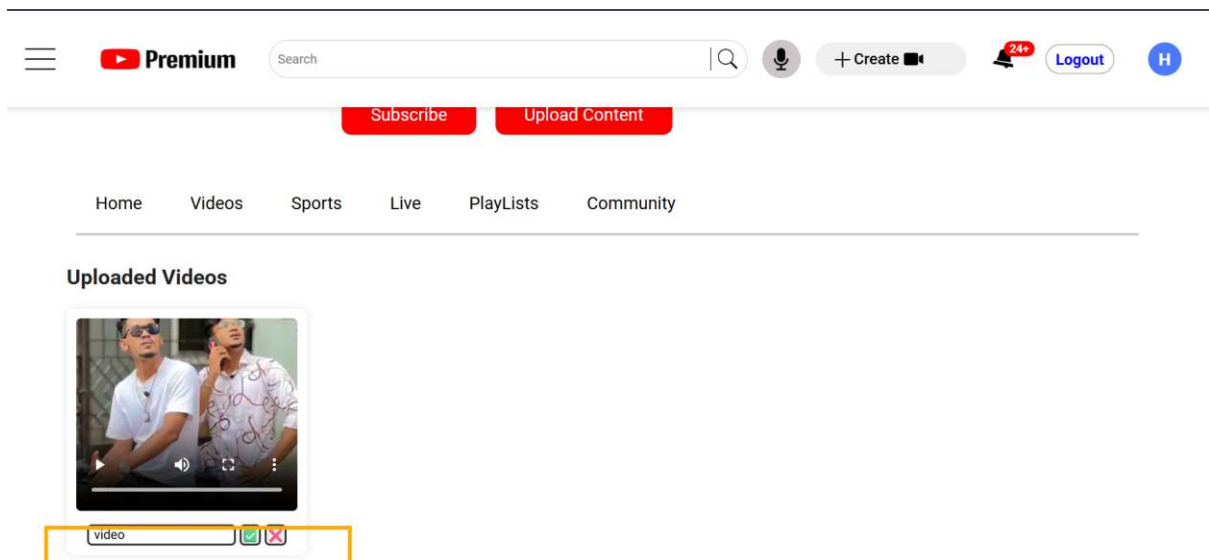
Home Videos Sports Live PlayLists Community

Users can see their uploaded videos

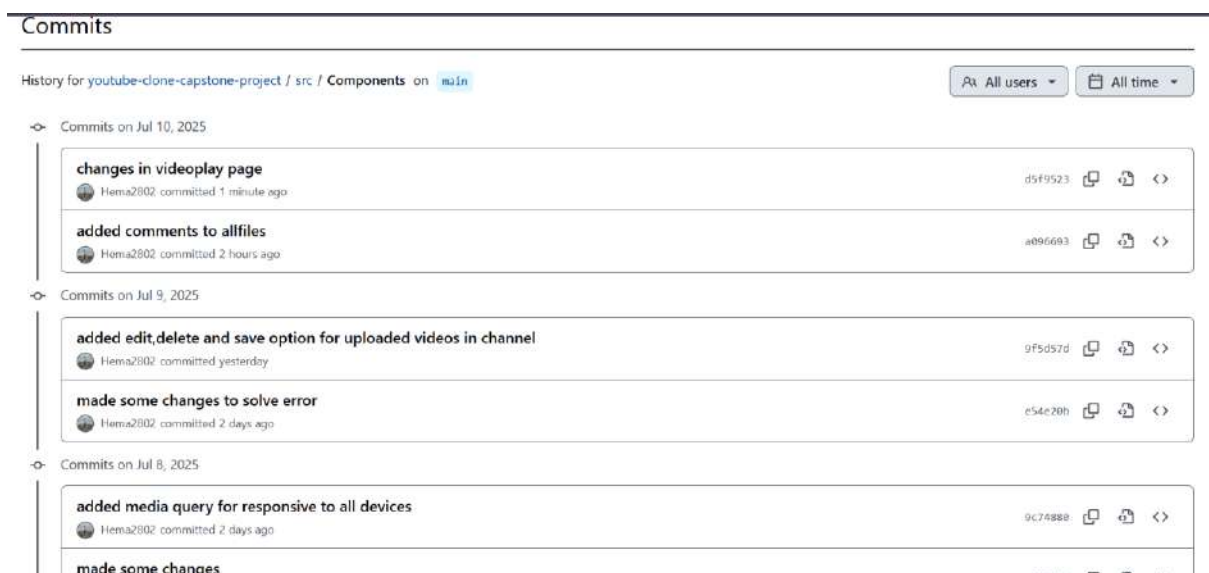


Users can edit, delete and save their changes in uploaded videos





Git version control system



Conclusion

This YouTube clone project demonstrates a strong implementation of core video-sharing platform features using React on the frontend and Node.js with Express on the backend, integrated with MongoDB for persistent data storage. The application emphasizes clean UI/UX, secure user management, and seamless media playback.

Front-End (React):

- The Home Page is designed with a clean layout, including a responsive NavBar, SideBar, video grid, and category filter buttons, ensuring accessibility across devices.
- User Authentication is handled through secure JWT tokens, supporting both registration and login workflows.
- The Video Player Page functions well, interactive like/dislike buttons, and a dynamic comment system with add/edit/delete support.
- The Channel Page allows users to create and manage their videos
- A functional search bar allows users to find videos by title efficiently.
- Category filters dynamically adjust the video feed, offering a smooth user experience for content discovery

Back-End (Node.js, Express, MongoDB):

- RESTful API endpoints are well-structured for handling users, videos, and comments.
- MongoDB is used effectively to store video metadata, user profiles, channel information, and comments.
- JWT-based authentication is integrated securely, protecting restricted routes and ensuring that only authenticated users can perform specific actions .

Final Thoughts:

This project effectively showcases full-stack development skills, from frontend design and routing logic to backend data management and security practices. It adheres to modern development standards, making it a solid foundation for a production-level video-sharing platform.

GITHUB LINK:

The screenshot shows the GitHub interface for the repository 'youtube-clone-capstone-project' by user Hema2802. The repository is public and has 360k/978 lines of code and 49 commits. The file list on the left includes 'Backend', 'public', 'src', '.gitignore', 'README.md', 'eslint.config.js', 'index.html', 'package-lock.json', 'package.json', and 'vite.config.js'. The right sidebar contains sections for 'About', 'Releases', 'Packages', and 'Languages'.

File	Commit Message	Time
Backend	added comments to allfiles	3 hours ago
public	added data	5 hours ago
src	changes in videoplay page	36 minutes ago
.gitignore	Initial commit - React Vite setup	2 weeks ago
README.md	Update README.md	now
eslint.config.js	Initial commit - React Vite setup	2 weeks ago
index.html	added Home folder and clear default code	2 weeks ago
package-lock.json	created controller,model and routes in backend and install...	last week
package.json	created controller,model and routes in backend and install...	last week
vite.config.js	Initial commit - React Vite setup	2 weeks ago

<https://github.com/Hema2802/youtube-clone-capstone-project/tree/main>